

Laborator 3- Algoritmi

1. Pentru un arbore binar de cautare, implementati operatiile de parcurgere a arborelui, de cautare a unui nod, precum si inserarea si stergerea unui nod.

Modul de implementare a unui arbore binar in C se face:

<pre>typedef struct nod_arb { int key; struct nod_arb *left, *right, *parent; } t_nod_arb; typedef struct { t_nod_arb *root; } t_arbore;</pre>	Obs: • Se definește în fișierul header • Declararea unei variabile arbore se face: t_arbore T; • Declararea unei variabile nod se face: t_nod_arb *n;
---	--

Pseudocod:

<pre>MAKE_ROOT(ARBORE A, KEY) Allocate memory for A.root A.root.key := KEY A.root.left := NULL A.root.right := NULL A.root.parent := NULL return A.root //returneaza referinta la //nodul radacina END MAKE_ROOT</pre>	<pre>INORDER_WALK(NOD ROOT) IF ROOT <> NULL INORDER_WALK(ROOT.left) PRINT ROOT.key INORDER_WALK(ROOT.right) END IF END INORDER_WALK</pre>
<pre>CREATE_NODE(KEY) Nod n Allocate memory for n n.key := KEY n.left := NULL n.right := NULL n.parent := NULL RETURN n END CREATE_NODE</pre>	<pre>PREORDER_WALK(NOD ROOT) IF ROOT <> NULL PRINT ROOT.key PREORDER_WALK(ROOT.left) PREORDER_WALK(ROOT.right) END IF END PREORDER_WALK</pre>
<pre>POSTORDER_WALK(NOD ROOT) IF ROOT <> NULL POSTORDER_WALK(ROOT.left) POSTORDER_WALK(ROOT.right) PRINT ROOT.key END IF END POSTORDER_WALK</pre>	<pre>TREE_MIN(NOD N) WHILE N.left <> NULL DO N := N.left END WHILE RETURN N END TREE_MIN</pre>

<pre> TREE_MAX(NOD N) WHILE N.right<> NULL DO N := N.right END WHILE RETURN N END TREE_MAX </pre>	<pre> TREE_SEARCH(NOD N, KEY) IF N == NULL OR KEY==N.key THEN RETURN N IF KEY < N.key THEN RETURN TREE_SEARCH(N.left,KEY) ELSE RETURN TREE_SEARCH(N.right, KEY) END IF END TREE_SEARCH </pre>
<pre> TREE_SUCCESOR(NOD N) NOD y IF (N.right <> NULL) RETURN TREE_MIN(N.right) END IF y := N.parent WHILE y<>NULL AND n== y.right DO N := y y := y.parent END WHILE RETURN y END TREE_SUCCESOR </pre>	<pre> IT_TREE_SEARCH(NOD N, KEY) WHILE N <> NULL AND KEY<>N.key DO IF (KEY < N.key) N := N.left ELSE N := N.right END IF END WHILE RETURN N END IT_TREE_SEARCH </pre>
<pre> TREE_INSERT(ARBORE A, NOD N) NOD y, NOD x y := NULL x := A.root WHILE x<>NULL DO y := x IF N.key < x.key x := x.left ELSE x := x.right END IF END WHILE N.parent := y IF y == NULL THEN A.root := N ELSE IF N.key < y.key THEN y.left := N ELSE y.right :=N END IF END TREE_INSERT </pre>	<pre> TREE_DELETE(ARBORE A, NOD N) NOD y //nodul N are 1 copil dreapta/stanga IF N.left ==NULL THEN TRANSPLANT(A, N, N.right) ELSE IF N.right == NULL THEN TRANSPLANT(A, N, N.left) ELSE //nodul N are 2 copii y := TREE_MIN(N.right) IF y.parent <> N THEN TRANSPLANT(A, y, y.right) y.right := N.right y.right.parent := y END IF TRANSPLANT(A, N, y) y.left := N.left y.left.parent := y END IF END TREE_DELETE </pre>

<pre> MAIN() ARBORE T NOD n, r, s PRINT "Nodul radacina cu cheia x= " READ x //nodul r este radacina arborelui r=MAKE_ROOT(T, x) PRINT "Introduceti x pana la citirea 0 \n" PRINT "x= " READ x WHILE x<>0 DO n := CREATE_NODE(x) TREE_INSERT(T, n) PRINT "x= " READ x END WHILE PRINT "IN-order walk:\n" INORDER_WALK(r) PRINT "\n Pre-order walk:\n" PREORDER_WALK(r) PRINT "\n Post-order walk:\n" POSTORDER_WALK(r) PRINT NEWLINE PRINT "Regasire nod cu cheia x= " READ x n := IT_TREE_SEARCH(r, x) IF n<>NULL THEN PRINT "Nodul cu cheia "+n.key+ "gasit iterativ\n" ELSE PRINT "Nodul nu a fost gasit iterativ\n" END IF n := TREE_SEARCH(r, x) IF n<>NULL THEN PRINT "Nodul cu cheia "+n.key+ "gasit recursiv\n" ELSE PRINT "Nodul nu a fost gasit recursiv\n" END IF n:= TREE_MIN(r) PRINT "Tree minimum: "+n.key n := TREE_MAX(r) PRINT "Tree maximum: "+n.key PRINT "Succesorul lui x=" Read x // continuare MAIN()→ </pre>	<pre> // continuare MAIN() n:= TREE_SEARCH(r,x) //succesorul nodului n va fi nodul s s:=TREE_SUCCESOR(n) IF s<>NULL THEN PRINT "Nod successor: "+s.key+"\n" ELSE PRINT "NULL \n" END IF PRINT "Stergere nod cu cheia x= " READ x n:= TREE_SEARCH(r, x) IF n<>NULL THEN TREE_DELETE(T, n) PRINT "Nod sters \n" INORDER_WALK(r) ELSE PRINT "\n Nod negasit!" END IF END MAIN </pre> <pre> TRANSPLANT(<i>ARBORE A, NOD U, NOD V</i>) IF u.parent == NULL THEN A.root := V ELSE IF U==U.parent.left THEN U.parent.left := V ELSE U.parent.right := V END IF IF V<>NULL THEN V.parent := U.parent END IF END TRANSPLANT </pre>
---	--

